

四、 实验三 寄存器堆实现

1 实验目的

1. 熟悉并掌握 MIPS 计算机中寄存器堆的原理和设计方法。
2. 初步了解 MIPS 指令结构和源操作数/目的操作数的概念。
3. 熟悉并运用 verilog 语言进行电路设计。
4. 为后续设计 cpu 的实验打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 学习 MIPS 计算机中寄存器堆的设计及原理，如：有多少个寄存器，有无特殊设置的寄存器，mips 指令如何去索引寄存器的等。
2. 自行设计本次实验的方案，画出结构框图，详细标出输入输出端口，本次实验建议设计为异步读同步写的寄存器堆，即读寄存器不需要时钟控制，但写寄存器需时钟控制。
3. 本次实验建议寄存器堆设计为 1 个写端口和 2 个读端口，后续 CPU 实验用到的寄存器堆需要 1 个写端口和 2 个读端口。
4. 根据设计的实验方案，使用 verilog 编写相应代码。
5. 对编写的代码进行仿真，得到正确的波形图。
6. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 4.1。外围模块中需调用封装好的 LCD 触摸屏模块，显示寄存器堆的读写端口地址和数据，最好能扫描出所有寄存器的值显示在 LCD 触摸屏上，并且需要利用触摸功能输入寄存器堆的读写地址和写数据。

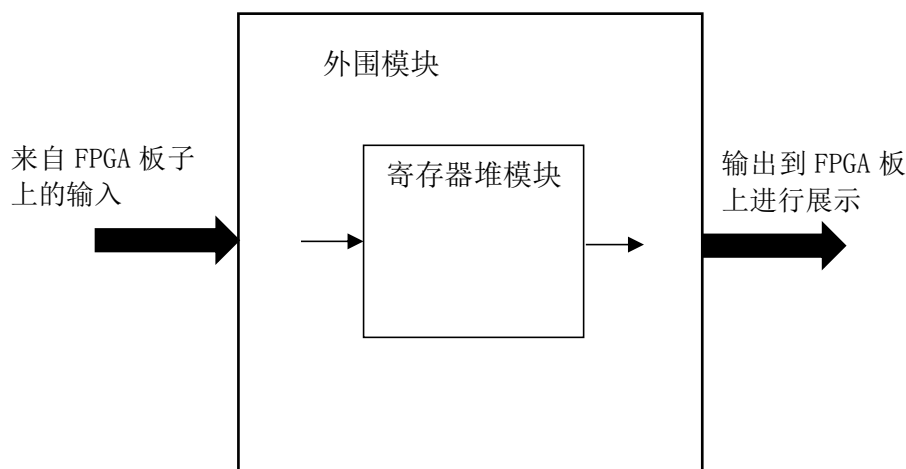


图 4.1 寄存器堆设计实验的顶层模块大致框图

7. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

4 实验要求

1. 做好预习：

- 1) 掌握寄存器堆的工作原理；
- 2) 确定寄存器堆的输入输出端口设计；
- 3) 在课前画好寄存器堆的设计框图或实验原理图；
- 4) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 4.1。

2. 实验实施：

- 1) 确认寄存器堆的设计框图的正确性；
- 2) 编写 verilog 代码；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料；
- 4) 完成调用寄存器堆模块的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示，按照检查人员的要求，对特定寄存器读写，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 调用 LCD 触摸屏

在寄存器堆实验中，调用 LCD 触摸屏的显示和输入功能时，有几点需要注意的地方。

5.1 显示功能

在本实验中需要扫描出寄存器堆里的所有寄存器的值显示在 LCD 触摸屏上，比如我们在 LCD 屏上 1~32 号显示块上对应显示 0~31 号寄存器的显示地址。由于 LCD 屏在显示时，是根据显示块号“display_number”来选择要显示的数据的，故可以给寄存器堆再增加一个调试读端口(此时寄存器堆有 3 个读端口)，该读端口的地址为 display_number-1，对应 0~31，读出的数据即为要显示的模块，代码可以如下编写：

```
1 assign test_addr = display_number-5'd1;
2 //-----{输出到触摸屏显示}begin
3 //根据需要显示的数修改此小节，
4 //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
5 //44 块显示区域从 1 开始编号，编号为 1~44，
6     always @(posedge clk)
```

```
7      begin
8          if (display_number > 6'd0 && display_number < 6'd33 )
9              begin //块号 1~32 显示 32 个通用寄存器的值
10                 display_valid <= 1'b1;
11                 display_name[39:16] <= "REG";
12                 display_name[15: 8] <= {4'b0011,3'b000,test_addr[4]};
13                 display_name[7 : 0] <= {4'b0011,test_addr[3:0]};
14                 display_value      <= test_data;
15             end
16         else
17             begin
18                 case(display_number)
19                     6'd33: //显示读端口 1 的地址
20                         begin
21                             display_valid <= 1'b1;
22                             display_name  <= "RADD1";
23                             display_value <= raddr1;
24                         end
25                     .....
26                     .....
```

test_addr 为寄存器堆新增的调试读端口，而 test_data 为通过该端口读出的数据

可以看到在显示 32 个寄存器时，其“display_name”域有特殊处理，其头 3 个字符显示“REG”，后两个字符则显示 16 进制的寄存器号“00~1F”，由于 0~9 字符的 ASCII 码为 48~57(8'b0011_0000~8'b0011_1001)，故“display_name”第 4 位字符(0 或 1)赋值{7'b0011_000,test_addr[4]}即可。

但对于第 5 位字符(0~F)的赋值则有稍许麻烦，因为 A~F 字符的 ASCII 编码不是随后的 58~63，即与 0~9 的编码不连续。但我们在设计 LCD 屏显示字符时，将内部的 ASCII 编码 58~63 也作为 A~F 字符的编码，即 ASCII 编码 48~ 63(8'b0011_0000 ~ 8'b0011_1111)对应字符 0~F，，故我们可以直接给第 5 位字符赋值{4'b0011,test_addr[3:0]}。

5.2 输入功能

在调用 LCD 触摸屏的输入功能时，其输入的数据为 32 位的数据，但在寄存器堆的实验中，读写地址为 5 位的，但依然可以使用屏输入读写地址，比如，可以取输入的 32 位数据的低 5 位作为读写地址。

6 参考设计

本部分给出的寄存器堆实验的参考设计方案中，共实现 32 个寄存器，其中 0 号寄存器读出的值恒为 0，寄存器堆为异步读同步写，共有 1 个写端口和 3 个读端口，其中一个读端口为调试读端口，用于扫描读出 32 个寄存器，给出了实验原理和设计框图，并提供源代码。

6.1 实验原理

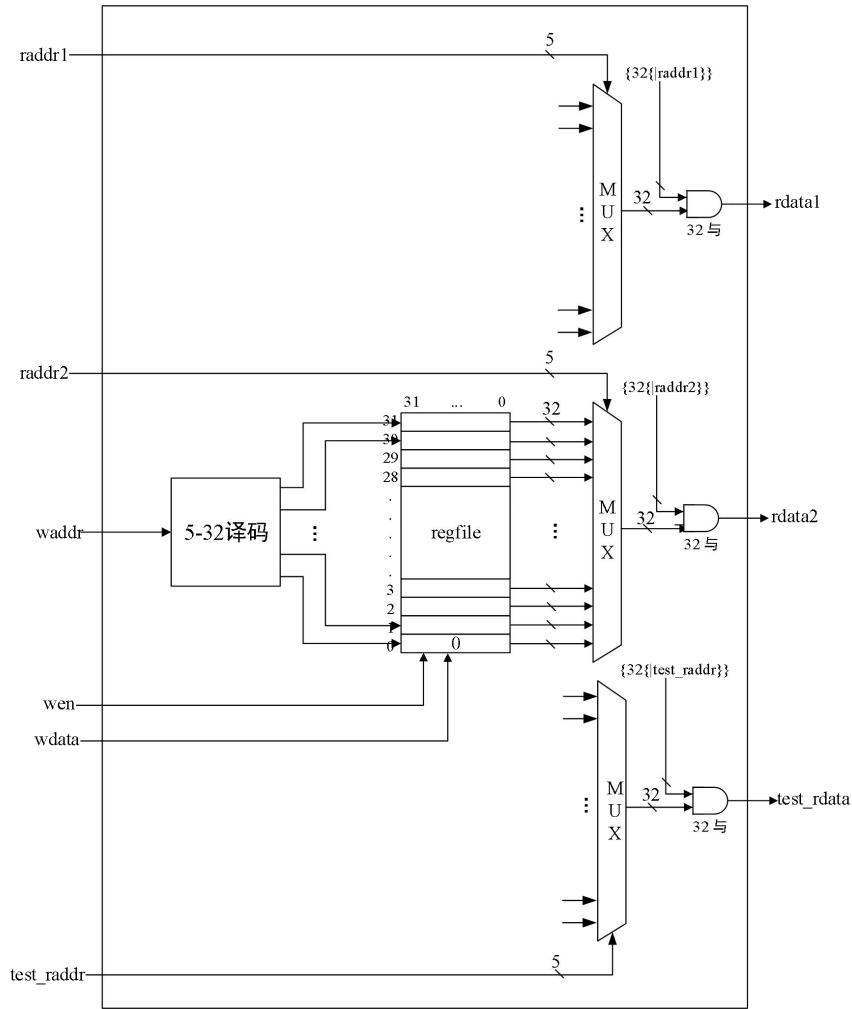


图 4.2 寄存器堆实验的原理图

6.2 实验顶层模块框图

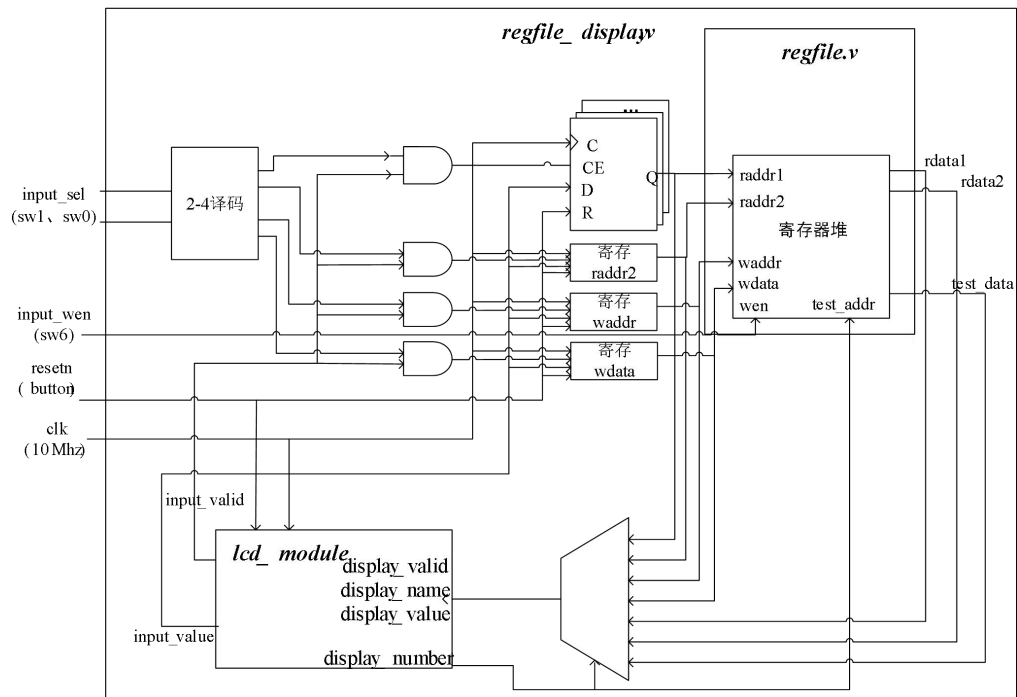


图 4.3 寄存器堆参考设计的顶层模块框图

regfile_display.v 源代码:

```
1  //*****
2  //  > 文件名: regfile_display.v
3  //  > 描述   : 寄存器堆显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
4  //  > 作者   : LOONGSON
5  //  > 日期   : 2016-04-14
6  //*****
7  module regfile_display(
8      //时钟与复位信号
9      input clk,
10     input resetn,    //后缀"n"代表低电平有效
11
12     //拨码开关, 用于产生写使能和选择输入数
13     input wen,
14     input [1:0] input_sel,
15
16     //led 灯, 用于指示写使能信号, 和正在输入什么数据
17     output led_wen,
18     output led_waddr,    //指示输入写地址
19     output led_wdata,    //指示输入写数据
20     output led_raddr1,    //指示输入读地址 1
21     output led_raddr2,    //指示输入读地址 2
22
23     //触摸屏相关接口, 不需要更改
24     output lcd_rst,
25     output lcd_cs,
26     output lcd_rs,
27     output lcd_wr,
28     output lcd_rd,
29     inout[15:0] lcd_data_io,
30     output lcd_bl_ctr,
31     inout ct_int,
32     inout ct_sda,
33     output ct_scl,
34     output ct_rstn
35 );
36 //----{LED 显示}begin
37     assign led_wen      = wen;
38     assign led_raddr1 = (input_sel==2'd0);
39     assign led_raddr2 = (input_sel==2'd1);
40     assign led_waddr  = (input_sel==2'd2);
41     assign led_wdata  = (input_sel==2'd3);
42 //----{LED 显示}end
43
44 //----{调用寄存器堆模块}begin
45     //寄存器堆多增加一个读端口, 用于在触摸屏上显示 32 个寄存器值
46     wire [31:0] test_data;
47     wire [4 :0] test_addr;
48
```

```
49     reg [4 :0] raddr1;
50     reg [4 :0] raddr2;
51     reg [4 :0] waddr;
52     reg [31:0] wdata;
53     wire [31:0] rdata1;
54     wire [31:0] rdata2;
55     regfile rf_module(
56         .clk      (clk      ),
57         .wen      (wen      ),
58         .raddr1   (raddr1),
59         .raddr2   (raddr2),
60         .waddr    (waddr   ),
61         .wdata    (wdata   ),
62         .rdata1   (rdata1),
63         .rdata2   (rdata2),
64         .test_addr(test_addr),
65         .test_data(test_data)
66     );
67 //-----{调用寄存器堆模块}end
68
69 //-----{调用触摸屏模块}begin-----//
70 //-----{实例化触摸屏}begin
71 //此小节不需要更改
72     reg      display_valid;
73     reg [39:0] display_name;
74     reg [31:0] display_value;
75     wire [5 :0] display_number;
76     wire      input_valid;
77     wire [31:0] input_value;
78
79     lcd_module lcd_module(
80         .clk      (clk      ), //10Mhz
81         .resetn   (resetn   ),
82
83         //调用触摸屏的接口
84         .display_valid (display_valid ),
85         .display_name  (display_name  ),
86         .display_value (display_value ),
87         .display_number (display_number),
88         .input_valid   (input_valid   ),
89         .input_value   (input_value   ),
90
91         //lcd 触摸屏相关接口，不需要更改
92         .lcd_rst      (lcd_rst      ),
93         .lcd_cs       (lcd_cs       ),
94         .lcd_rs       (lcd_rs       ),
95         .lcd_wr       (lcd_wr       ),
96         .lcd_rd       (lcd_rd       ),
97         .lcd_data_io  (lcd_data_io  ),
98         .lcd_bl_ctr   (lcd_bl_ctr   ),
99         .ct_int       (ct_int       ),
```

```
100         .ct_sda          (ct_sda          ),
101         .ct_scl          (ct_scl          ),
102         .ct_rstn         (ct_rstn         )
103     );
104 //-----{实例化触摸屏}end
105
106 //-----{从触摸屏获取输入}begin
107 //根据实际需要输入的数修改此小节,
108 //建议对每一个数的输入, 编写单独一个 always 块
109     //32 个寄存器显示在 7~38 号的显示块, 故读地址为 (display_number-1)
110     assign test_addr = display_number-5'd7;
111     //当 input_sel 为 2'b00 时, 表示输入数为读地址 1, 即 raddr1
112     always @(posedge clk)
113     begin
114         if (!resetn)
115         begin
116             raddr1 <= 5'd0;
117         end
118         else if (input_valid && input_sel==2'd0)
119         begin
120             raddr1 <= input_value[4:0];
121         end
122     end
123
124     //当 input_sel 为 2'b01 时, 表示输入数为读地址 2, 即 raddr2
125     always @(posedge clk)
126     begin
127         if (!resetn)
128         begin
129             raddr2 <= 5'd0;
130         end
131         else if (input_valid && input_sel==2'd1)
132         begin
133             raddr2 <= input_value[4:0];
134         end
135     end
136
137     //当 input_sel 为 2'b10 时, 表示输入数为写地址, 即 waddr
138     always @(posedge clk)
139     begin
140         if (!resetn)
141         begin
142             waddr <= 5'd0;
143         end
144         else if (input_valid && input_sel==2'd2)
145         begin
146             waddr <= input_value[4:0];
147         end
148     end
149
150     //当 input_sel 为 2'b11 时, 表示输入数为写数据, 即 wdata
```

```
151     always @(posedge clk)
152     begin
153         if (!resetn)
154             begin
155                 wdata <= 32'd0;
156             end
157         else if (input_valid && input_sel==2'd3)
158             begin
159                 wdata <= input_value;
160             end
161     end
162 //-----{从触摸屏获取输入}end
163
164 //-----{输出到触摸屏显示}begin
165 //根据需要显示的数修改此小节,
166 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
167 //44 块显示区域从 1 开始编号, 编号为 1~44,
168     always @(posedge clk)
169     begin
170         if (display_number >6'd6 && display_number <6'd39 )
171             begin //块号 7~38 显示 32 个通用寄存器的值
172                 display_valid <= 1'b1;
173                 display_name[39:16] <= "REG";
174                 display_name[15: 8] <=
175                 {4'b0011,3'b000,test_addr[4]};
176                 display_name[7 : 0] <= {4'b0011,test_addr[3:0]};
177                 display_value <= test_data;
178             end
179         else
180             begin
181                 case(display_number)
182                     6'd1 : //显示读端口 1 的地址
183                         begin
184                             display_valid <= 1'b1;
185                             display_name <= "RADD1";
186                             display_value <= raddr1;
187                         end
188                     6'd2 : //显示读端口 1 读出的数据
189                         begin
190                             display_valid <= 1'b1;
191                             display_name <= "RDAT1";
192                             display_value <= rdata1;
193                         end
194                     6'd3 : //显示读端口 2 的地址
195                         begin
196                             display_valid <= 1'b1;
197                             display_name <= "RADD2";
198                             display_value <= raddr2;
199                         end
200                     6'd4 : //显示读端口 2 读出的数据
201                         begin
```



```
201             display_valid <= 1'b1;
202             display_name  <= "RDATA2";
203             display_value <= rdata2;
204         end
205         6'd5 : //显示写端口的地址
206         begin
207             display_valid <= 1'b1;
208             display_name  <= "WADDR";
209             display_value <= waddr;
210         end
211         6'd6 : //显示写端口写入的数据
212         begin
213             display_valid <= 1'b1;
214             display_name  <= "WDATA";
215             display_value <= wdata;
216         end
217         default :
218         begin
219             display_valid <= 1'b0;
220             display_name  <= 40'd0;
221             display_value <= 32'd0;
222         end
223     endcase
224 end
225 end
226 //-----{输出到触摸屏显示}end
227 //-----{调用触摸屏模块}end-----//
228 endmodule
```

7 实验报告要求

1. 完善regfile.v代码，在注释位置完善代码部分，将代码或截图放在实验报告中添加源文件的位置。
2. 功能仿真：对完善好的regfile.v进行功能仿真，验证读写端口和测试端口功能。对指定寄存器写入数据，观察读端口1、读端口2和测试端口的数据写入波形，将正确波形和仿真代码截图放到实验报告功能仿真部分。（对2个指定寄存器读、写即可，生成两组波形，可以分为两个波形截图。）
3. 绑定引脚：和之前要求一样，参照引脚对应关系，截图放到添加约束文件部分。
4. 了解MIPS架构中寄存器，有多少个寄存器？有无特殊设置的寄存器？MIPS指令如何索引寄存器？三个问题答案放在实验报告最后。